CSCI 4100

Operating Systems and Architecture

Chapter 6: Multi-Object Synchronization

Topics

- 6.0 – Multi-Object Synchronization
- 6.1 – Multiprocessor Lock Performance
- 6.2 – Lock Design Patterns
- 6.4 – Multi-Object Atomicity
- 6.5 – Deadlock

Multi-Object Synchronization

Coordinating access to multiple shared objects presents additional challenges.

- The design of shared objects can have a large impact on multiprocessor performance.
- Splitting shared objects into separate objects each with its own lock can improve performance.
- Correctness may be impacted by interactions between multiple shared objects.
- Multiple locks raises the possibility of deadlock, where threads are stuck waiting for each other in a cycle.

Multiprocessor Lock Performance

- Client-server applications exhibit **request parallelism**, where individual requests can be processed in parallel.
- Parallelism may not improve throughput as much as expected, due to the following issues:
  1. Locks can create bottlenecks, preventing threads from executing concurrently
  2. Shared data on multiprocessor systems must be copied from one processor cache to another.
  3. Hardware is subject to **false sharing** where multiple data structures with different sharing behavior are stored in the same cache entry.

Lock Design Patterns

If locks become bottlenecks, a system can be redesigned to improve performance.

- **Fine-grained locking** partitions a shared object's state with each partition guarded by a lock.
- **Per-processor data structures** partition an object's state so accesses to a partition are made by threads running on the same processor.
- In the **ownership design pattern**, a shared object is removed from a shared container so a single thread can read or modify it.
- In a **staged architecture** the system is divided into multiple stages, so that only threads within a stage can access the stage's shared data.

Fine-Grained Locking

- If a single lock is a bottleneck, a shared object can be broken into multiple partitions, each with its own lock.
- With multiple locks, operations that access data across partitions may be difficult to implement.
- An RWLock can be used to guard the entire shared object
  - Operations performed on a single partition can acquire the RWLock in read mode for parallel access to partitions.
  - Operations performed across partitions can acquire the RWLock in write mode for exclusive access to all partitions.
- All partition locks can be acquired before performing a complex operation.
- The partitions can be made large enough so that operations like resizing a data structure can be performed on a single partition.

Per-Processor Data Structures

- A shared object can be broken into partitions based on the number of processors on the machine.
- A per-processor ready list can make sure threads are scheduled to run on the same processor across context switches.
- This approach improves performance by reducing the need to copy data from one processor cache to another.
- There is still the possibility that data in one processor cache will be needed by a thread running on a different processor.

Ownership Design Pattern

- In the ownership design pattern, a thread removes an object from a container and can operate exclusively on the object without a lock.
- Consider a multithreaded web browser that processes webpage objects in three stages:
  1. The network stage receives the object from the network.
  2. The parse stage parses and transforms the object so it can be rendered.
  3. The render stage renders the object to the screen.
- Queues are used between stages to hold the objects.
- Multiple threads can run in each stage, each one processing a different object.

Staged Architecture

- A staged architecture divides a system into multiple subsystems called stages, each with its own private data and threads.
- Stages communicate using shared producer-consumer queues.
- Breaking up computation into stages improves modularity, so that each stage can be developed and tested separately.
- Staged architecture can also improve cache locality.
- Event-driven programming is a form of staged architecture with one thread per stage.
- If one stage becomes a bottleneck it can be allocated more threads.

Multi-Object Atomicity

- Individual shared objects can guarantee atomic operations.
- An operation that involves multiple shared objects is not guaranteed to be atomic.
- Consider a balance transfer between two accounts, each of which is a shared object:
  1. Subtract the amount from account A.
  2. Add the amount to account B.
- Each operation is atomic, but the entire transfer is not.
- In between the two operations, the system is in an inconsistent state.

Careful Class Design

- Sometimes the problem of managing multiple shared objects can be solved through the design of classes and interfaces.
- Consider an implementation of the Too Much Milk problem with the following classes:
  - A `Note` class with `readNote` and `writeNote` methods.
  - A `Fridge` class with `checkForMilk` and `addMilk` methods.
- To guarantee that a thread can atomically check for milk and write a note, we could refactor the Fridge class to use the following methods:
  - `checkForMilkAndSetNoteIfNeeded`
  - `addMilk`

Acquire-All/Release-All

- Acquire-all/release-all is a more general technique for managing multiple shared objects.
  1. A thread makes a request to perform a group of operations.
  2. Before executing the operations, the thread acquires the locks for all of the shared objects involved.
  3. Once the locks are acquired, the operations can be performed.
  4. Finally, the locks are all released.
- Multiple threads can run concurrently if their requests are non-overlapping.
- The set of all requests are **serializable**, which means that executing them in parallel is equivalent to executing them sequentially.
- This approach requires knowing which locks will be needed before executing the request.

Two-Phase Locking

- Two-Phase locking is a variation on acquire-all/release-all where locks can be acquired as needed.
  - In the first phase locks can be acquired at any time, but not released.
  - In the second phase locks can be released, but not acquired.
- Like acquire-all/release-all, two-phase locking is serializable.
- The additional flexibility means that threads do not have to acquire locks they don't need or hold them longer than necessary.
- Unlike acquire-all/release-all, two-phase locking can lead to deadlock.

Deadlock

- A **deadlock** occurs when every thread in a set of threads is waiting for an action that can only be performed by another thread in the group.
- Example: Two Threads (A, B) and Two Locks (1, 2):
  1. Thread A tries to acquire lock 1 and succeeds.
  2. Thread B tries to acquire lock 2 and succeeds.
  3. Thread A tries to acquire lock 2 and is blocked.
  4. Thread B tries to acquire lock 1 and is blocked.
- Thread A is blocked waiting for thread B to release lock 2.
- Thread B is blocked waiting for thread A to release lock 1.
- Neither thread can make progress until the other releases a lock.
- The threads are deadlocked.

The Dining Philosophers

Example: **Dining Philosophers**

- Five philosophers are sitting down at a meal at a round table.
- Between each philosopher is a chopstick.
- Each one needs the chopsticks on either side in order to start eating.
- A philosopher takes the following steps to eat:
  1. Pick up the left chopstick.
  2. Pick up the right chopstick.
  3. Start eating.
- If all of the philosophers pick up their left chopstick at the same time, they will all be waiting for their right neighbor to put down their right chopstick.

Deadlock vs. Starvation

- Deadlock is a particular kind of starvation.
- **Starvation** occurs when a thread fails to make progress for an indefinite period of time.
- Example: in a RWLock readers can starve if writers are always given priority when trying to acquire the lock.
- Just because a system can suffer starvation or deadlock doesn't mean it will.
- Typically, starvation and deadlock only occur in certain interleavings of actions, much like race conditions.

Necessary Conditions for Deadlock

There are four necessary conditions for deadlock.

1. **Bounded resources** – a finite number of threads can simultaneously access a resource.
2. **No preemption** – once a thread acquires a resource, its ownership cannot be revoked until the thread acts to release it.
3. **Wait while holding** – a thread holds one resource while waiting for another.
4. **Circular waiting** – there is a set of waiting threads such that each thread is waiting for a resource held by another.

Necessary Conditions – Dining Philosophers

The Dining Philosophers met all four conditions:

1. **Bounded resources** – each chopstick can be held by one philosopher at a time.
2. **No preemption** – a philosopher does not put down a chopstick until she is done eating.
3. **Wait while holding** – a philosopher can hold the left chopstick while waiting for the right chopstick
4. **Circular waiting** – each philosopher is waiting for their right neighbor to release their right fork, creating a cycle of waiting.

Dealing with Deadlock

There are three approaches to handling a deadlock:

1. **Prevention** - exploit or limit the behavior of the program to eliminate one of the necessary conditions of deadlock.
2. **Avoidance** - if we know that an action may lead to deadlock, delay the action until it may be performed safely.
3. **Detection and Recovery** - detect when deadlock occurs, then recover by undoing actions that lead to the deadlock.

Preventing Deadlock (1)

How do we prevent deadlock in the Dining Philosophers problem?

- Option 1: Provide sufficient resources.
  - Add an additional chopstick
  - Allow any philosopher to use any chopstick.
  - One philosopher will always be able to acquire two chopsticks.
- Option 2: Preempt resources.
  - Have a waiter available to watch for a deadlock situation.
  - If a deadlock is about to occur, the waiter takes a chopstick away from one philosopher and gives it to another one.

Preventing Deadlock (2)

- Option 3: Release a resource before waiting to acquire another one.
  - If a philosopher has picked up the left chopstick, but their neighbor has the right chopstick, they must put down the left chopstick and try again.
  - Deadlock is prevented
  - Starvation is still possible if all of the philosophers are stuck picking up and putting down their left chopsticks.
- Option 4: Number all resources and require they be acquired in order.
  - Each philosopher must pick up the lower-numbered chopstick first.
  - Deadlock is prevented
  - It is possible that only one philosopher can eat at a time even though at least two should be able to do so.

Avoiding Deadlock

- If we can not remove any of the necessary conditions for deadlock, we can still avoid deadlock by making threads wait until it is safe to perform an action.
- First, we classify the states that a system can be in:
  - In a **safe state**, for any possible set of resource requests there is at least one ordering of requests that does not lead to deadlock.
  - In an **unsafe state**, there is at least one set of resource requests that leads to deadlock regardless of the order.
  - In a **deadlocked state**, the system has at least one deadlock.

The Banker's Algorithm

The **Banker's Algorithm** avoids deadlock as follows:

- All threads declare up front the maximum resources they need.
- Requests are granted as long as they keep the system in a safe state.
- If granting a request would put the system in an unsafe state, the request is delayed until this is no longer the case.
- If there is some ordering of requests that avoids deadlock, the Banker's algorithm will find such an ordering.

Example: Page Allocation

- A system has 8 pages of memory and three processes: A, B, and C.
- A needs 4 pages, B and C both need 5 pages.
- If the processes take turns requesting a single page, the system will deadlock after the 8th page is allocated.
- If the system uses the Banker's Algorithm, the system will delay B and C when they request their third page, allowing A to acquire all of its remaining pages.
- After A releases its pages, the allocation continues until B has 4 pages, at which point C is delayed until B acquires its fifth page.
- After B releases its pages, C is free to acquire its remaining pages and release them.

Example: Dining Philosophers

- The system is in a safe state if a philosopher has acquired both chopsticks or there is still a chopstick on the table.
- If all philosophers request their left chopstick, four of the requests will be granted (say philosophers 0, 1, 2 and 3 requesting chopsticks 0, 1, 2, and 3), but the fifth (philosopher 4 requesting chopstick 4) will be delayed.
- The remaining four philosophers then request their right chopsticks, but only one of the requests (philosopher 0 requesting chopstick 4) is granted.
- When that philosopher has finished eating, each of the other philosophers can have their requests granted as the chopsticks become available.

Detecting and Recovering From Deadlocks

Preventing deadlocks may require a significant amount of overhead, so many systems allow them to occur.

This approach requires two additional mechanisms:

1. A way to recover from deadlock with minimal harm to users.
2. A way to detect when deadlock occurs.

Strategies for Deadlock Recovery

- The system can give the user the option to kill a deadlocked process.
  - If the process is not making progress anyway, termination is a reasonable alternative.
  - If any of process's threads holds a lock on a shared kernel object, the system could be left in an inconsistent state.
- The program can continue without the requested resource.
  - Web services are often designed to be resilient when resources are unavailable.
  - Deadlocks are rare and hard to test for, so error conditions must be handled carefully.
- Operations can be performed using transactions.
  - If acquiring a lock leads to a deadlock, the system can rollback or undo the thread's actions, so other threads can proceed.
  - When the other threads complete, the thread can be restarted and may proceed as if the deadlock never occurred.
  - Transactions are atomic, so the intermediate state is only visible to the thread performing the transaction.

Strategies for Deadlock Detection

- Assume a deadlock has occurred if an operation takes too much time.
  - If a router is overloaded it will start dropping packets instead of making the sender wait until room is available.
  - This approach is simple, but not very precise.
- Construct a graph where the threads and resources are nodes.
  - Add a directed edge between each acquired resource and the thread that has acquired it.
  - Add a directed edge between each waiting thread and the resource it is waiting on.
  - If the resulting graph contains a cycle, the system is deadlocked.
- Use a variation on the Banker's Algorithm.
  - The maximum number of resources requested by each thread is unknown.
  - The system checks if pending requests can be satisfied assuming no further requests are made.
  - If this is not possible, there is a deadlock.